Impact of interference on an IEEE 802.11b Wi-Fi link

An ns-3 study of two co-located transmitter and receiver pairs: how far the interfering pair must be before the primary link stops paying for it, and why the worst place to put it is not the closest one.

NAME	ROLL NO.	SIMULATOR	DATE
_____	_____	ns-3.44, C++20	02 Nov 2026

01 Aim

Build an ns-3 simulation with two wireless links in the same area, one carrying the traffic we care about and one generating interfering traffic. Configure both with IEEE 802.11b, run UDP over each, capture packets with PCAP, measure the packet delivery of the primary link, and repeat while changing the distance between the primary pair and the interfering pair.

Task	What was asked	Where it is done
1	Two pairs of Wi-Fi nodes (TX + RX)	`nodes.Create(4)`, Fig. 1, Section 3
2	Configure devices with IEEE 802.11b	`WIFI_STANDARD_80211b`, DSSS 11 Mbit/s, ad hoc MAC
3	Position nodes to study interference	`ListPositionAllocator`, link budget in Table 2
4	UDP from transmitter to its receiver	`UdpClient` on A, `UdpServer` on B (port 9)
5	Second transmission that interferes	C to D on the same channel (port 10), Section 3
6	PCAP tracing	`phy.EnablePcap()` with radiotap headers, Section 6
7	Measure packet delivery under interference	FlowMonitor + MAC retry traces, Sections 5 and 7
8	Repeat for different distances	31 distances x 5 seeds, plus a load sweep, Section 7

02 Background

802.11b PHY. 802.11b runs in the 2.4 GHz band with 22 MHz channels and offers 1, 2, 5.5 and 11 Mbit/s using DSSS and CCK. Every station on the same channel shares one collision domain, so a second link nearby is not "noise" in the abstract; it is a competitor for the same air.

CSMA/CA. Before sending, a station listens. If it decodes a preamble or sees energy above its threshold, it defers, then counts down a random backoff. Each unicast frame must be acknowledged; if the ACK

does not come, the sender doubles its contention window and tries again, up to the retry limit (`FrameRetryLimit` = 7 attempts in ns-3.44). This is the mechanism that decides who wins, so it shapes every result below.

Three distances that matter. The *transmission range* is where a frame can be decoded. The *carrier-sense range* is where a station hears enough to defer. The *interference range* is where a signal is strong enough to ruin someone else's reception. When the interfering transmitter is outside the sender's sensing range but inside the receiver's, CSMA/CA has nothing to work with. That is the classic **hidden-terminal** problem, and it turned out to be the most damaging region in this experiment.

03 Simulation design

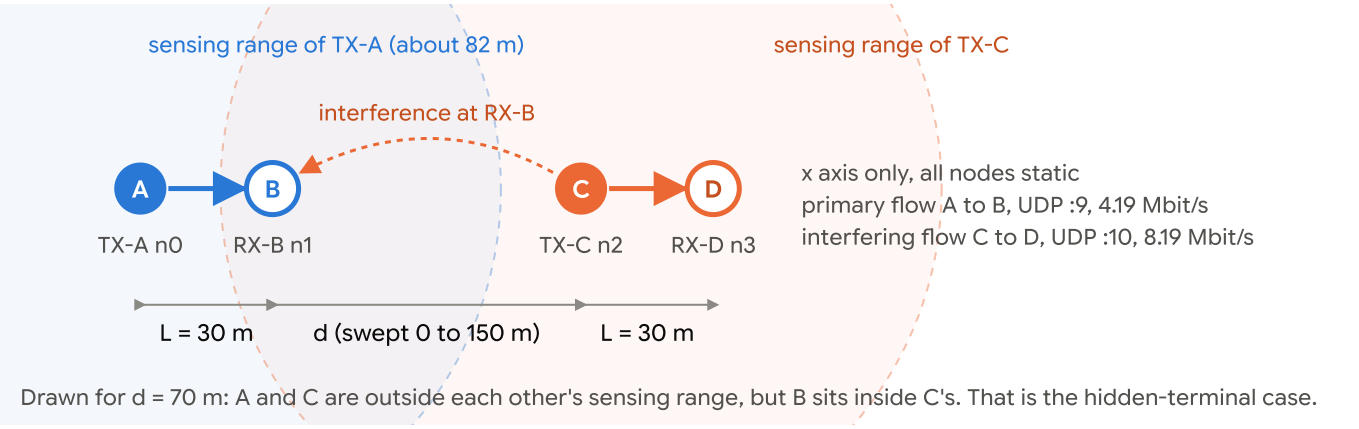


Fig. 1. Topology. Both pairs share one ad hoc 802.11b channel. Only *d*, the gap between the primary receiver B and the interfering transmitter C, is changed between runs.

Table 1. Parameters

Parameter	Value
Standard / MAC	802.11b, `AdhocWifiMac`
Rate control	Constant, DSSS 11 Mbit/s data, 1 Mbit/s control
Tx power	16 dBm
Rx sensitivity	-82 dBm
Propagation	Log-distance, n = 3, 40.05 dB at 1 m
Packet size	1024 B UDP payload
Primary load	1 pkt / 2 ms = 4.19 Mbit/s
Interferer load	1 pkt / 1 ms = 8.19 Mbit/s
Traffic window	1 s to 10 s (9 s)
Seeds per point	5 (RngRun 1 to 5)

Table 2. Link budget

$P_{rx}(r) = 16 - 40.05 - 30 \log_{10}(r)$ dBm

Path	r (m)	P_{rx} (dBm)
A to B (wanted)	30	-68.4
C to B, d = 20	20	-63.1
C to B, d = 50	50	-75.0
C to B, d = 70	70	-79.4
A to C, d = 52	82	-81.5
C to B, d = 90	90	-82.7

The sweep located the effective sensing edge between 82 m and 83 m, which is within half a dB of the configured -82 dBm.

Why these numbers. The primary link at 30 m has about 25 dB of margin over the noise floor, so on its own it should lose nothing, and the baseline confirms it. The interferer is deliberately saturated at 8.19

Mbit/s, which is more than an 802.11b channel can carry at 1024 B frames (around 5.7 Mbit/s of goodput). That way it always has a frame waiting and the result shows the worst case rather than a lucky gap in its traffic. RTS/CTS is left off, which is the default and how most real 802.11b networks ran.

> **Two artifacts I had to remove before the numbers meant anything.** (1) Both senders start at exactly t = 1 s, so their ARP request broadcasts collided. Broadcasts are never retried, ns-3 marks the ARP entry dead for 100 s, and the whole flow silently delivered 0 packets in some seeds. The ARP caches are now pre-filled with `NeighborCacheHelper`. (2) Two CBR sources started at the same instant stay in lock-step forever, so they either always collide or never do. The interferer now starts with a random offset of 1 to 1000 µs per run.

---

04 Source code

The complete program, `wifi-interference.cc`. Each numbered comment block maps to a task from Section 1.

```
/*
 * Lab Assignment 10: impact of co-channel interference on an 802.11b link.
 *
 *   n0 (TX-A) ----L---- n1 (RX-B) ----d---- n2 (TX-C) ----L---- n3 (RX-D)
 *   |<---- primary link ---->|              |<--- interfering link --->|
 *
 * Primary flow: n0 -> n1 (UDP, port 9)
 * Interfering flow: n2 -> n3 (UDP, port 10), switched on with --interf=1
 * d (distance between primary receiver and interfering transmitter) is swept.
 */
#include "ns3/applications-module.h"
#include "ns3/core-module.h"
#include "ns3/flow-monitor-module.h"
#include "ns3/internet-module.h"
#include "ns3/mobility-module.h"
#include "ns3/network-module.h"
#include "ns3/wifi-module.h"

#include <iomanip>
#include <iostream>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("WifiInterferenceLab");

static uint32_t g_retries = 0;    // failed data attempts at TX-A (no ACK)
static uint32_t g_finalFail = 0;  // frames discarded by TX-A after retry limit

static void
TxDataFailed(Mac48Address)
{
    g_retries++;
}

static void
TxFinalFailed(Mac48Address)
{
    g_finalFail++;
}

int
main(int argc, char* argv[])
{
    double linkDist = 30.0;     // m, TX-A to RX-B and TX-C to RX-D
    double interfDist = 50.0;   // m, RX-B to TX-C
    bool interf = true;         // enable the interfering pair
    double simTime = 10.0;      // s
    uint32_t pktSize = 1024;    // bytes
    double primaryInt = 0.002;  // s between primary packets  (~4 Mbit/s)
    double interfInt = 0.001;   // s between interferer packets (~8 Mbit/s, saturating)
```

```
    double rxSens = -82.0;      // dBm, receiver sensitivity / carrier sense
    bool pcap = false;
    bool verbose = false;
    uint32_t run = 1;

    CommandLine cmd(__FILE__);
    cmd.AddValue("linkDist", "TX to RX distance of each pair (m)", linkDist);
    cmd.AddValue("interfDist", "Distance RX-B to TX-C (m)", interfDist);
    cmd.AddValue("interf", "Enable interfering transmission", interf);
    cmd.AddValue("simTime", "Simulation time (s)", simTime);
    cmd.AddValue("pktSize", "UDP payload size (bytes)", pktSize);
    cmd.AddValue("primaryInt", "Primary inter-packet interval (s)", primaryInt);
    cmd.AddValue("interfInt", "Interferer inter-packet interval (s)", interfInt);
    cmd.AddValue("rxSens", "PHY RxSensitivity (dBm)", rxSens);
    cmd.AddValue("pcap", "Write PCAP traces", pcap);
    cmd.AddValue("verbose", "Print a readable summary", verbose);
    cmd.AddValue("run", "RNG run number", run);
    cmd.Parse(argc, argv);

    RngSeedManager::SetSeed(7);
    RngSeedManager::SetRun(run);

    // ---- 1. Nodes: two transmitter/receiver pairs --------------------------
    NodeContainer nodes;
    nodes.Create(4); // 0 = TX-A, 1 = RX-B, 2 = TX-C, 3 = RX-D

    // ---- 2. 802.11b PHY + MAC (ad hoc, fixed 11 Mbit/s) --------------------
    YansWifiChannelHelper channel;
    channel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
    channel.AddPropagationLoss("ns3::LogDistancePropagationLossModel",
                               "Exponent", DoubleValue(3.0),
                               "ReferenceLoss", DoubleValue(40.046)); // 2.4 GHz @ 1 m

    YansWifiPhyHelper phy;
    phy.SetChannel(channel.Create());
    phy.Set("TxPowerStart", DoubleValue(16.0));
    phy.Set("TxPowerEnd", DoubleValue(16.0));
    phy.Set("RxSensitivity", DoubleValue(rxSens));
    phy.SetPcapDataLinkType(WifiPhyHelper::DLT_IEEE802_11_RADIO);

    WifiHelper wifi;
    wifi.SetStandard(WIFI_STANDARD_80211b);
    wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                                 "DataMode", StringValue("DsssRate11Mbps"),
                                 "ControlMode", StringValue("DsssRate1Mbps"));

    WifiMacHelper mac;
    mac.SetType("ns3::AdhocWifiMac");
    NetDeviceContainer devices = wifi.Install(phy, mac, nodes);

    // ---- 3. Positions ------------------------------------------------------
    MobilityHelper mobility;
    Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator>();
    pos->Add(Vector(0.0, 0.0, 0.0));                                // TX-A
    pos->Add(Vector(linkDist, 0.0, 0.0));                           // RX-B
    pos->Add(Vector(linkDist + interfDist, 0.0, 0.0));              // TX-C
    pos->Add(Vector(2 * linkDist + interfDist, 0.0, 0.0));          // RX-D
    mobility.SetPositionAllocator(pos);
    mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
    mobility.Install(nodes);

    // ---- Internet stack ----------------------------------------------------
    InternetStackHelper stack;
    stack.Install(nodes);
    Ipv4AddressHelper addr;
    addr.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer ifs = addr.Assign(devices);

    // Pre-fill ARP caches. Otherwise the two ARP broadcasts sent at t=1 s can
    // collide (broadcasts are never retried) and a dead ARP entry silently
    // blocks the whole flow for 100 s, which has nothing to do with the study.
    NeighborCacheHelper neighbors;
    neighbors.PopulateNeighborCache();

    // ---- 4. Primary UDP flow  TX-A -> RX-B ---------------------------------
    UdpServerHelper primaryServer(9);
    ApplicationContainer srvA = primaryServer.Install(nodes.Get(1));
```

```
    srvA.Start(Seconds(0.5));
    srvA.Stop(Seconds(simTime + 1.0));

    UdpClientHelper primaryClient(ifs.GetAddress(1), 9);
    primaryClient.SetAttribute("MaxPackets", UintegerValue(0xFFFFFFFF));
    primaryClient.SetAttribute("Interval", TimeValue(Seconds(primaryInt)));
    primaryClient.SetAttribute("PacketSize", UintegerValue(pktSize));
    ApplicationContainer cliA = primaryClient.Install(nodes.Get(0));
    cliA.Start(Seconds(1.0));
    cliA.Stop(Seconds(simTime));

    // ---- 5. Interfering UDP flow  TX-C -> RX-D ------------------------------
    Ptr<UdpServer> interfSrvPtr;
    if (interf)
    {
        UdpServerHelper interfServer(10);
        ApplicationContainer srvC = interfServer.Install(nodes.Get(3));
        srvC.Start(Seconds(0.5));
        srvC.Stop(Seconds(simTime + 1.0));
        interfSrvPtr = DynamicCast<UdpServer>(srvC.Get(0));

        UdpClientHelper interfClient(ifs.GetAddress(3), 10);
        interfClient.SetAttribute("MaxPackets", UintegerValue(0xFFFFFFFF));
        interfClient.SetAttribute("Interval", TimeValue(Seconds(interfInt)));
        interfClient.SetAttribute("PacketSize", UintegerValue(pktSize));
        ApplicationContainer cliC = interfClient.Install(nodes.Get(2));
        // random phase offset so the two CBR sources are not in lock-step
        Ptr<UniformRandomVariable> jitter = CreateObject<UniformRandomVariable>();
        cliC.Start(Seconds(1.0) + MicroSeconds(jitter->GetInteger(1, 1000)));
        cliC.Stop(Seconds(simTime));
    }

    // ---- 6. PCAP tracing ----------------------------------------------------
    if (pcap)
    {
        phy.EnablePcap("lab10-interf", devices);
    }

    // MAC-level counters on the primary transmitter only
    std::string sm = "/NodeList/0/DeviceList/0/$ns3::WifiNetDevice/RemoteStationManager/";
    Config::ConnectWithoutContext(sm + "MacTxDataFailed", MakeCallback(&TxDataFailed));
    Config::ConnectWithoutContext(sm + "MacTxFinalDataFailed", MakeCallback(&TxFinalFailed));

    // ---- 7. Measurement -----------------------------------------------------
    FlowMonitorHelper fmHelper;
    Ptr<FlowMonitor> monitor = fmHelper.InstallAll();

    Simulator::Stop(Seconds(simTime + 1.0));
    Simulator::Run();

    monitor->CheckForLostPackets();
    Ptr<Ipv4FlowClassifier> classifier =
        DynamicCast<Ipv4FlowClassifier>(fmHelper.GetClassifier());

    double active = simTime - 1.0;
    uint64_t pTx = 0, pRx = 0, iTx = 0, iRx = 0;
    double pDelay = 0, pThr = 0, iThr = 0;
    for (auto& [id, st] : monitor->GetFlowStats())
    {
        Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(id);
        double thr = st.rxBytes * 8.0 / active / 1e6;
        if (t.destinationPort == 9)
        {
            pTx = st.txPackets;
            pRx = st.rxPackets;
            pThr = thr;
            pDelay = st.rxPackets ? st.delaySum.GetSeconds() / st.rxPackets * 1e3 : 0;
        }
        else if (t.destinationPort == 10)
        {
            iTx = st.txPackets;
            iRx = st.rxPackets;
            iThr = thr;
        }
        if (verbose)
        {
            std::cout << "Flow " << id << "  " << t.sourceAddress << ":" << t.sourcePort
```

```
                      << " -> " << t.destinationAddress << ":" << t.destinationPort << "\n"
                      << "  Tx packets : " << st.txPackets << "\n"
                      << "  Rx packets : " << st.rxPackets << "\n"
                      << "  Lost       : " << st.txPackets - st.rxPackets << "\n"
                      << std::fixed << std::setprecision(2)
                      << "  PDR        : " << (st.txPackets ? 100.0 * st.rxPackets / st.txPackets : 0)
                      << " %\n"
                      << "  Throughput : " << thr << " Mbit/s\n"
                      << "  Mean delay : "
                      << (st.rxPackets ? st.delaySum.GetSeconds() / st.rxPackets * 1e3 : 0)
                      << " ms\n";
            if (t.destinationPort == 9)
            {
                std::cout << "  MAC retries at TX-A  : " << g_retries << "\n"
                          << "  Retry-limit drops    : " << g_finalFail << "\n";
            }
        }
    }

    double pdr = pTx ? 100.0 * pRx / pTx : 0.0;
    double ipdr = iTx ? 100.0 * iRx / iTx : 0.0;
    // CSV: interfDist,interf,run,pTx,pRx,pdr,thr,delay,iTx,iRx,ipdr,ithr,retries,finalFail
    std::cout << std::fixed << std::setprecision(3) << "CSV," << interfDist << "," << interf
              << "," << run << "," << pTx << "," << pRx << "," << pdr << "," << pThr << ","
              << pDelay << "," << iTx << "," << iRx << "," << ipdr << "," << iThr << ","
              << g_retries << "," << g_finalFail << std::endl;

    if (pcap)
    {
        monitor->SerializeToXmlFile("lab10-flowmon.xml", true, true);
    }
    Simulator::Destroy();
    return 0;
}
```

Sweep script

Runs the distance sweep (31 points x 5 seeds, plus 5 baseline runs with the interferer off) and the interferer-load sweep at three representative distances. Seeds for one point run in parallel; the whole thing takes about a minute.

```
#!/bin/bash
# Experiment 1: distance sweep, 5 RNG runs per point
echo "dist,interf,run,pTx,pRx,pdr,thr,delay,iTx,iRx,ipdr,ithr,retries,finalFail" > dist.csv
for r in 1 2 3 4 5; do ./wifi-interference --interf=0 --run=$r | grep CSV | cut -d, -f2- >> dist.csv; done
for d in $(seq 0 5 150); do
  for r in 1 2 3 4 5; do
    ./wifi-interference --interfDist=$d --run=$r | grep CSV | cut -d, -f2- >> dist.csv &
  done; wait
done
# Experiment 2: interferer load sweep at three representative distances
echo "dist,interval,run,pTx,pRx,pdr,thr,delay,iTx,iRx,ipdr,ithr,retries,finalFail" > load.csv
for d in 20 70 120; do for it in 0.016 0.008 0.004 0.003 0.002 0.0015 0.001; do
  for r in 1 2 3; do
    (./wifi-interference --interfDist=$d --interfInt=$it --run=$r | grep CSV | cut -d, -f2- | awk -F, -v it=$it
'BEGIN{OFS=","}{$2=it; print}') >> load.csv &
  done; wait
done; done
```

05 Build and run

terminal · build

```
$ N=$HOME/ns3/install          # ns-3.44 install prefix (headers + lib64)
$ g++ -std=c++20 -O2 -I$N/include wifi-interference.cc -o wifi-interference \
      -L$N/lib64 $(ls $N/lib64/libns3.44-*.so | sed -E 's#.*/lib(.*)\.so#-l\1#') \
      -Wl,-rpath,$N/lib64
$ ./wifi-interference --PrintHelp | head -14
wifi-interference [Program Options] [General Arguments]

Program Options:
    --linkDist:    TX to RX distance of each pair (m) [30]
    --interfDist:  Distance RX-B to TX-C (m) [50]
    --interf:      Enable interfering transmission [true]
    --simTime:     Simulation time (s) [10]
    --pktSize:     UDP payload size (bytes) [1024]
    --primaryInt:  Primary inter-packet interval (s) [0.002]
    --interfInt:   Interferer inter-packet interval (s) [0.001]
    --rxSens:      PHY RxSensitivity (dBm) [-82]
    --pcap:        Write PCAP traces [false]
    --verbose:     Print a readable summary [false]
    --run:         RNG run number [1]
```

terminal · four representative runs

```
$ ./wifi-interference --interf=0 --verbose=1
Flow 1  10.1.1.1:49153 -> 10.1.1.2:9
  Tx packets : 4500
  Rx packets : 4500
  Lost       : 0
  PDR        : 100.00 %
  Throughput : 4.21 Mbit/s
  Mean delay : 1.03 ms
  MAC retries at TX-A  : 0
  Retry-limit drops    : 0
$ ./wifi-interference --interfDist=20 --verbose=1
Flow 1  10.1.1.1:49153 -> 10.1.1.2:9
  Tx packets : 4500
  Rx packets : 3186
  Lost       : 1314
  PDR        : 70.80 %
  Throughput : 2.98 Mbit/s
  Mean delay : 457.91 ms
  MAC retries at TX-A  : 201
  Retry-limit drops    : 0
Flow 2  10.1.1.3:49153 -> 10.1.1.4:10
  Tx packets : 9000
  Rx packets : 3543
  Lost       : 5457
  PDR        : 39.37 %
  Throughput : 3.31 Mbit/s
  Mean delay : 485.72 ms
$ ./wifi-interference --interfDist=70 --verbose=1
Flow 1  10.1.1.1:49153 -> 10.1.1.2:9
  Tx packets : 4500
  Rx packets : 2895
  Lost       : 1605
  PDR        : 64.33 %
  Throughput : 2.71 Mbit/s
  Mean delay : 473.06 ms
  MAC retries at TX-A  : 1289
  Retry-limit drops    : 5
Flow 2  10.1.1.3:49153 -> 10.1.1.4:10
  Tx packets : 9000
  Rx packets : 6056
  Lost       : 2944
  PDR        : 67.29 %
  Throughput : 5.66 Mbit/s
  Mean delay : 467.66 ms
$ ./wifi-interference --interfDist=120 --verbose=1
Flow 1  10.1.1.1:49153 -> 10.1.1.2:9
  Tx packets : 4500
  Rx packets : 4500
  Lost       : 0
  PDR        : 100.00 %
  Throughput : 4.21 Mbit/s
  Mean delay : 1.03 ms
  MAC retries at TX-A  : 0
  Retry-limit drops    : 0
Flow 2  10.1.1.3:49153 -> 10.1.1.4:10
  Tx packets : 9000
  Rx packets : 6095
  Lost       : 2905
  PDR        : 67.72 %
  Throughput : 5.70 Mbit/s
  Mean delay : 467.28 ms
```

Flow 2 loses packets even when it is far away (d = 120 m). That is expected: it offers 8.19 Mbit/s into a channel that carries about 5.7, so its own queue overflows. It is the primary flow that we are judging.

06 PCAP analysis

`phy.EnablePcap("lab10-interf", devices)` writes one capture per node with a radiotap header (`DLT_IEEE802_11_RADIO`), so every 802.11 frame each radio decoded is visible, including ACKs and retransmissions. The PCAP runs were shortened to 4 s (3 s of traffic) to keep files small.

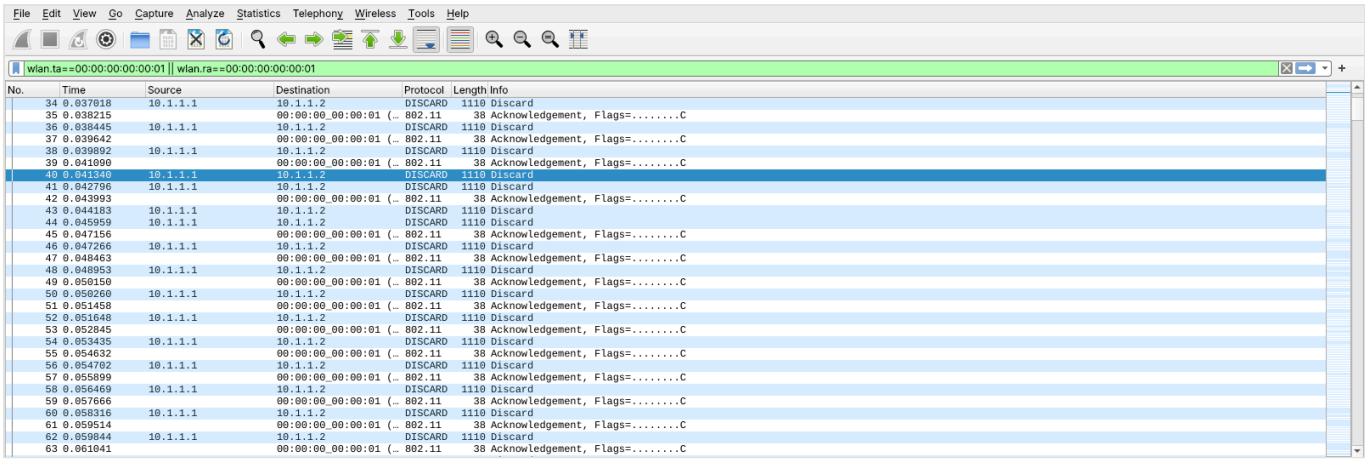


Fig. 2. Wireshark, TX-A's capture, d = 70 m, filtered on frames to or from A. Normal exchanges alternate data then ACK. Frames 40 to 41 and 43 to 44 are data frames sent back to back with no ACK between them: B never acknowledged frame 40, so A retransmitted it.

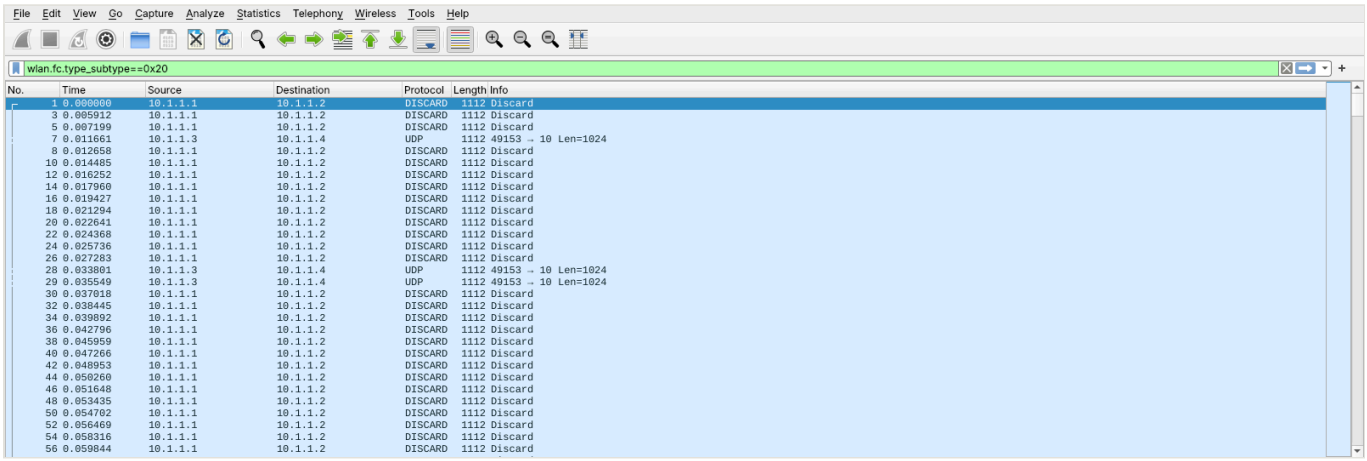


Fig. 3. Wireshark, RX-B's capture, data frames only. B decodes the interferer's frames (10.1.1.3 to 10.1.1.4) mixed in with its own traffic. B hears C; A does not. This is the evidence for the hidden-terminal reading of the 53 to 82 m region.

terminal · counting frames with tshark

```
$ ./wifi-interference --interfDist=70 --pcap=1 --simTime=4
$ ls *.pcap
lab10-interf-0-0.pcap  lab10-interf-1-0.pcap  lab10-interf-2-0.pcap  lab10-interf-3-0.pcap
$ # data frames put on air by TX-A (seen in TX-A's own capture)
$ tshark -r lab10-interf-0-0.pcap -Y "wlan.fc.type_subtype==0x20 && wlan.ta==00:00:00:00:00:01" | wc -l
1511
$ # ACKs that came back to TX-A
$ tshark -r lab10-interf-0-0.pcap -Y "wlan.fc.type_subtype==0x1d && wlan.ra==00:00:00:00:00:01" | wc -l
1059
$ # data frames from TX-C that RX-B decoded (B can hear C, A cannot)
$ tshark -r lab10-interf-1-0.pcap -Y "wlan.fc.type_subtype==0x20 && wlan.ta==00:00:00:00:00:03" | wc -l
667
```

Table 3. Frame accounting from the captures

Scenario	Data frames sent by A	ACKs back to A	MAC retries (trace)	C frames decoded at B
No interferer	1500	1500	0	0
d = 20 m (shared)	1249	1172	77	1234
d = 70 m (hidden)	1511	1059	452	667
d = 120 m (far)	1500	1500	0	0

The bookkeeping closes exactly in every row: *frames sent = ACKs received + MAC retries*. So the PCAP files and the MAC trace sources agree with each other, which is a cheap but useful check that the measurement code is counting what it claims to. Note the two interference cases fail in opposite ways. At 20 m A sends *fewer* frames than its application produced (1249 of 1500) because it keeps deferring to C. At 70 m A sends *more* frames than it had (1511) because it never defers but keeps having to retransmit.

07 Results

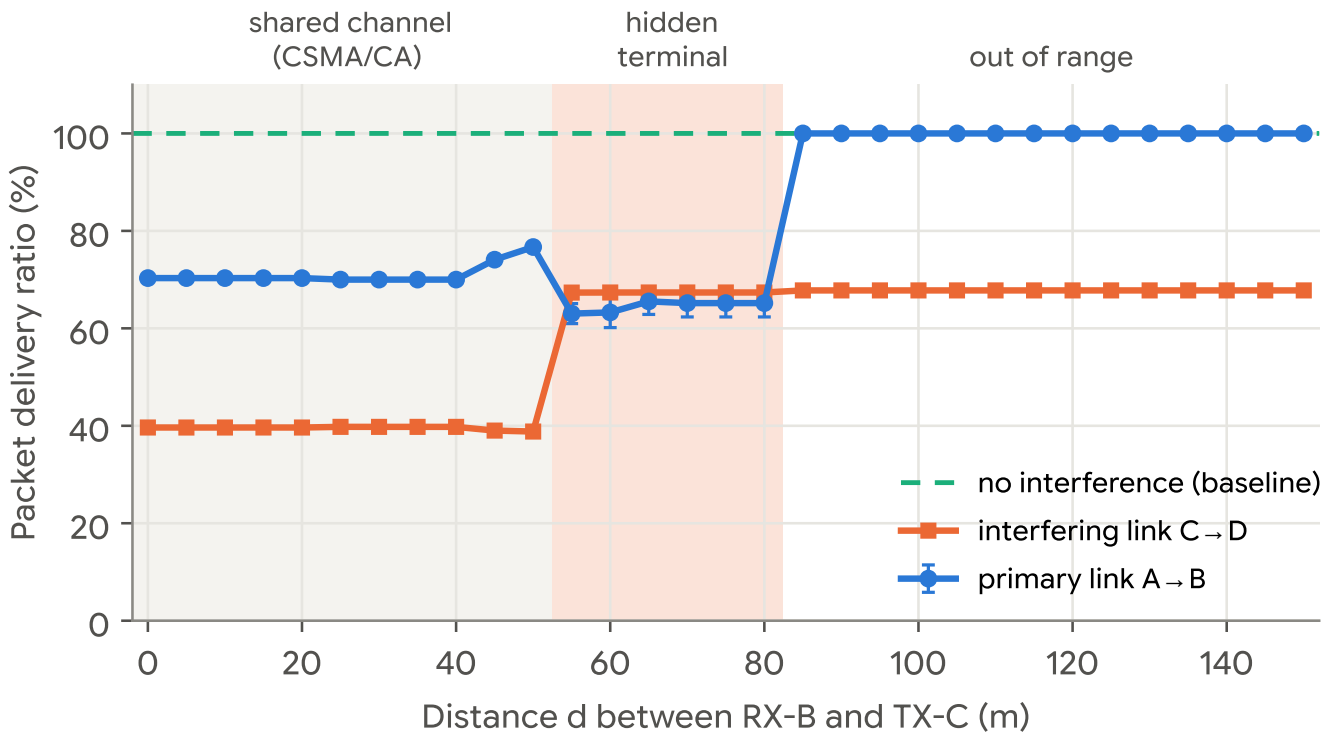


Fig. 4. Packet delivery ratio against d. Mean of 5 seeds, error bars are one standard deviation. Shaded band is the hidden-terminal region.

Table 4. Primary link at selected distances (mean of 5 seeds)

d (m)	Regime	PDR %	Goodput Mbit/s	Delay ms	MAC retries	Retry-limit drops	Interferer Mbit/s
none	baseline	100.0	4.21	1.0	0	0	0.00
0 to 20	shared	70.3 ± 0.4	2.96	460	198	0	3.34
45	shared	74.1 ± 0.6	3.12	455	100	0	3.28
50	shared	76.7 ± 0.4	3.23	452	29	0	3.27
55	hidden	63.0 ± 2.1	2.65	469	1329	4.2	5.67
70	hidden	65.2 ± 2.8	2.74	467	1297	3.8	5.67
85	far	100.0	4.21	1.0	0	0	5.70
120	far	100.0	4.21	1.0	0	0	5.70

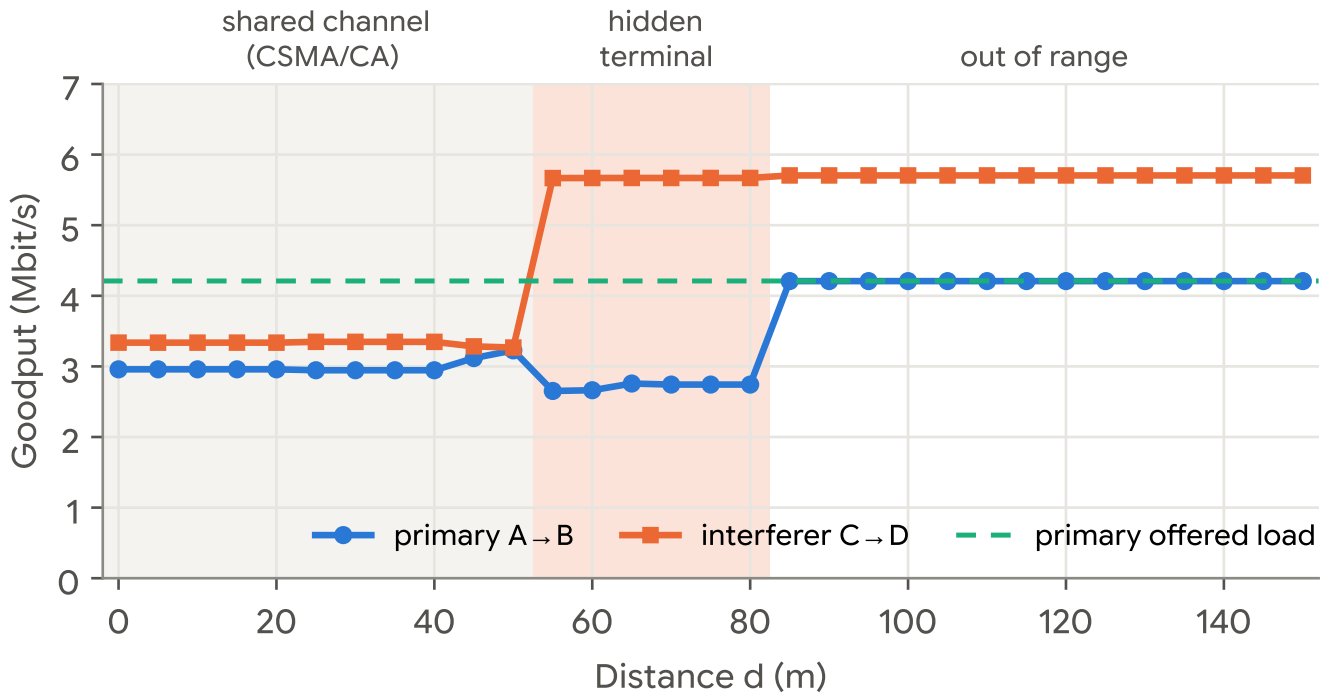


Fig. 5. Goodput of both flows. In the shared region they split the channel roughly evenly. In the hidden region the interferer takes the whole channel because it no longer defers to A.

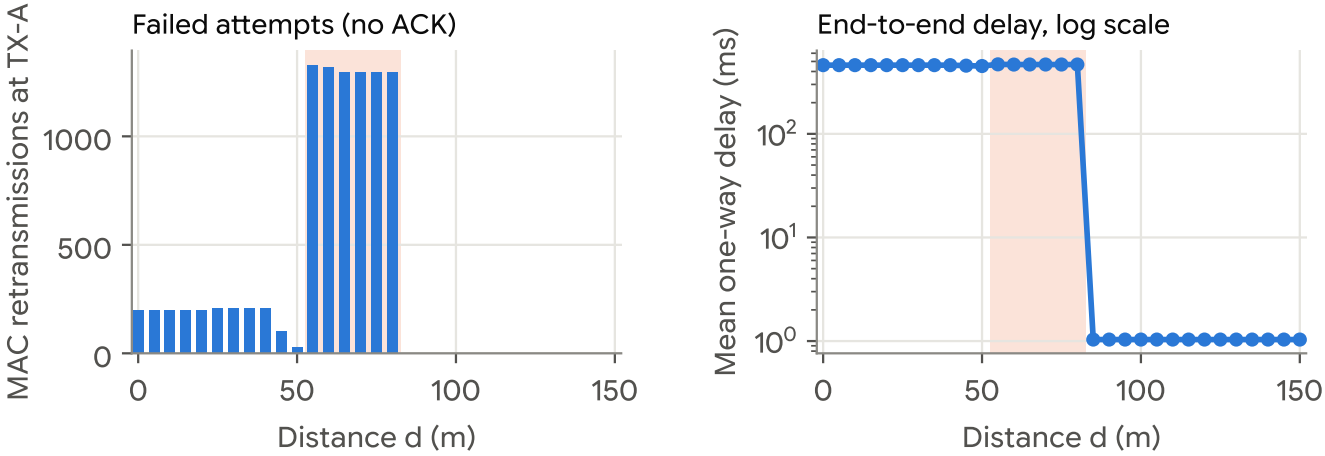


Fig. 6. Left: MAC-layer failed attempts at TX-A, which jump about 6x when C becomes hidden. Right: mean delay of delivered packets on a log scale, about 1 ms without contention and about 460 ms with it.

Interferer load at fixed distance

To check that the shape of Fig. 4 depends on how busy the interferer is, the interferer's rate was swept at three distances (3 seeds each), with the primary held at 4.19 Mbit/s.

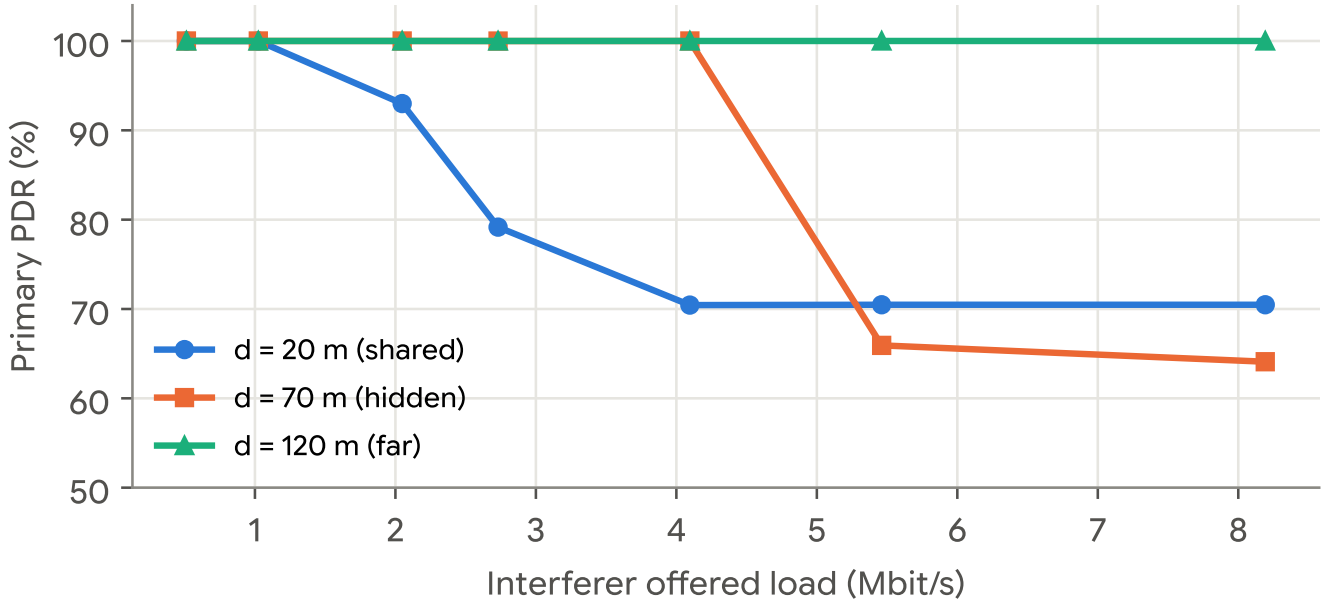


Fig. 7. Primary PDR against interferer offered load. At 120 m the interferer is harmless at any rate. At 20 m losses begin once the two loads together exceed what the channel can carry. At 70 m the link survives up to about 4 Mbit/s and then collapses.

---

08 Analysis

The curve in Fig. 4 is not monotonic, and that is the main finding. Moving the interferer away does not steadily help. There are three regimes with sharp edges, and the edges line up with the 82 m sensing range from the link budget.

```
d (m)  0 ............ 52 | 53 ........... 82 | 83 ............ 150
       |  A hears C      |  A can't hear C   |  B can't hear C
       |  C hears A      |  B still hears C  |
       |  CSMA/CA shares |  HIDDEN TERMINAL  |  no interaction
       |  the channel    |  collisions at B  |  (spatial reuse)
```

Regime 1: shared channel (d ≤ 52 m)

A and C hear each other, so CSMA/CA does its job: they take turns. Collisions are rare (about 200 retries over 9 s), and no frame ever hits the retry limit. The primary still loses about 30% of its packets, but not because of corrupted frames. It simply gets about half the airtime (2.96 of the roughly 6.3 Mbit/s the two flows carry together) while asking for 4.19 Mbit/s. The excess sits in A's MAC queue until it is dropped, which is why delivered packets average around 460 ms of delay. ns-3's `WifiMacQueue` drops any packet older than `MaxDelay` = 500 ms (and holds at most 500 packets), so the delay plateau just under that value is the queue limit, not the radio.

Within this regime PDR *rises* slightly from 70% to 77% between 40 m and 52 m, and retries fall from about 200 to 29. When A and C pick the same backoff slot they collide at B. At d = 20 m the interferer arrives at B 5 dB stronger than A, so the collision always destroys A's frame. At d = 50 m A is 6.6 dB stronger than C at B, and B can decode A's frame through the collision. This is the capture effect, and it is visible in the retry counter before it shows in PDR.

Regime 2: hidden terminal (53 m ≤ d ≤ 82 m)

This is the worst place for the interferer, even though it is farther away than in regime 1. A and C can no longer hear each other, so both transmit whenever they like. C is saturated, so it is on air almost all the time. B is still inside C's range, so whenever C's frame reaches B first, B locks onto it (Fig. 3 shows B decoding C's traffic) and A's frame is lost. A sees no ACK, doubles its backoff and tries again. With seed 1, retries go from 8 at d = 52 m to 1322 at d = 53 m. One metre, crossing the sensing edge, changes the MAC's life by two orders of magnitude.

The signal-to-interference ratio at B is actually healthy here (11 dB at d = 70 m, Table 2). The loss is not caused by C being loud. It is caused by B being *busy* with C's frame when A's arrives. That is why the curve is flat across the whole region instead of improving gradually as C moves away.

Two further points are easy to miss. First, the primary still loses most of its packets in its own queue, not on the air: only about 4 frames per run hit the retry limit, while about 1600 packets are dropped from the queue because retransmissions and doubled backoffs stretch the service time. Second, the network as a whole carries more in this region (2.74 + 5.67 = 8.4 Mbit/s) than in regime 1 (6.3 Mbit/s), because C now gets spatial reuse. The gain all goes to the interferer and the cost all lands on the primary. The standard deviation across seeds is also 4 to 7 times larger here, since outcomes depend on how often C's frames happen to start before A's.

Regime 3: out of range (d ≥ 83 m)

C's signal at B drops below the sensitivity threshold. B never syncs to it, and its energy is too small to hurt A's 25 dB margin. The primary is back at 100% delivery, 4.21 Mbit/s and about 1 ms delay, identical to the baseline, while the interferer runs at full rate. Both links coexist on one channel without any coordination.

What the load sweep adds

Fig. 7 shows the hidden terminal is a threshold problem, not a gradual one. At d = 70 m with the interferer at or below 4 Mbit/s, the primary delivers everything; MAC retries recover the occasional collision (891 retries at 2.7 Mbit/s, all recovered, 6 ms delay). The link only breaks once C's airtime fraction is high enough that retries keep running into it. In the shared region the decline is smooth and starts earlier, because there the cost is airtime, and every extra bit the interferer sends is taken out of the primary's share.

---

09 Conclusion

- Without interference the 802.11b link at 30 m delivers 100% of 4.19 Mbit/s with about 1 ms delay.
- A saturated interferer within carrier-sense range cuts primary PDR to about 70%. The loss comes from sharing airtime, and frames on air rarely fail.
- The worst case is a hidden interferer (53 to 82 m here), giving 63 to 65% PDR, 6x more retransmissions and the highest variance. Moving the interferer away made things worse before it made them better.
- Past the receiver's sensing range (83 m and beyond) the interference vanishes completely and both links run at full rate.
- The PCAP captures confirm the mechanism directly: missing ACKs and back-to-back retransmissions from A, and C's frames decoded at B. Frame counts from the captures match the MAC trace sources exactly.

Limitations. Static nodes on a line, a deterministic log-distance channel with no fading, a single fixed rate with no rate adaptation, and no RTS/CTS. Enabling RTS/CTS (`RtsCtsThreshold` set to 0) is the

obvious next experiment. It should mostly fix regime 2, because C would hear B's CTS and defer, at the cost of extra overhead in the other regimes.